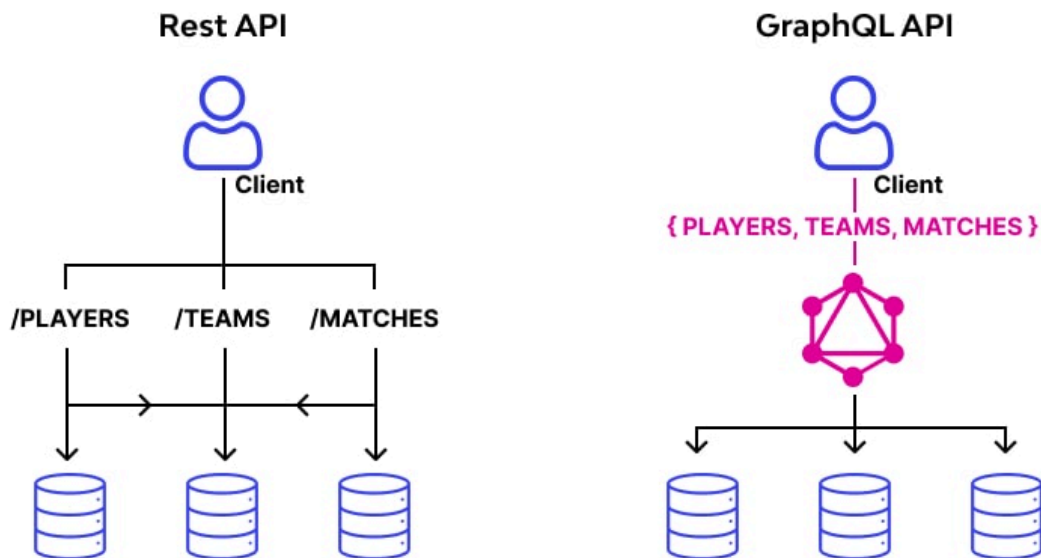


Both GraphQL and REST are popular API architectures that enable data exchange between different services or applications in a client-server model.



APIs facilitate data access and data manipulation, such as:

1. A client sends an API request to one or more endpoints on a server.
2. The server provides a response containing data, data status, or error codes.

REST and GraphQL allow you to create, modify, update, and delete data on a separate application, service, or module via APIs. APIs developed with REST are called API RESTful or API REST. The API developed with GraphQL is simply...API GraphQL. Frontend and backend teams use these API architectures to create modular and accessible applications. Using API architectures makes the system secure, modular, and scalable. API architectures also lead to higher performance and easier integration with other systems.

Next, we will discuss some other similarities between GraphQL and REST.

GraphQL vs REST Showdown

	GraphQL	REST
Performance	Fast	Multiple network calls take up more time
Query complexity	Query can become very complex because of different client's request	Since there are separate endpoints for different queries. The queries are simple
Popularity	Still Growing	Very Popular
Resources & community support	Growing	Large
Learning curve	Steep learning curve	Very simple learning curve
File uploading	No	Yes
Web caching	Uses web library	Inbuilt
Recommended use case	Multiple microservices and mobile apps	Simple apps and resource driven applications

Architecture

Both REST and GraphQL implement several common API architecture principles. For example, they both share the following principles:

- Neither is stateful, so the server doesn't save a response history between requests.
- Both use a client-server model, so requests from a single client will receive responses from a single server.
- Both are based on HTTP, as HTTP is the fundamental communication protocol.

Resource-driven design

Both REST and GraphQL design their data exchange around resources. A resource is understood to be any data or object that a client can access and manipulate through an API. Each resource has its own unique identifier (URI) and a set of operations (HTTP methods) that the client can perform on it.

For example, consider a social media API where users create and manage posts. In a resource-based API, a post would be a resource. It has its own unique identifier, for example: `/posts/1234`. At the same time, it has a set of operations, such as `GET` to retrieve posts in REST or `query(Query)` to retrieve posts in GraphQL.

Data exchange

Both REST and GraphQL support the same data formats.

JSON is the most common data exchange format, understood by all languages, platforms, and systems. The server returns JSON data to the client. Other data formats are supported but are less commonly used, including XML and HTML.

Similarly, both REST and GraphQL support caching capabilities. Therefore, clients and servers can cache frequently accessed data to speed up communication.

Language and database neutrality

Both GraphQL and REST APIs work with any database structure and programming language, both client-side and server-side. Therefore, these APIs are highly interoperable with any application.

What limitations of REST does GraphQL attempt to overcome?

GraphQL emerged in 2012 to address the need for speed in emerging social media platforms. Developers realized that existing API architectures, such as REST, were too structured and lengthy to create news feeds efficiently.

Next, we will discuss some of the challenges these APIs face.

Exchange of fixed-structure data

REST APIs require client requests to follow a fixed structure in order to receive resources. This rigid structure is easy to use, but it's not always the most efficient means of exchanging exactly the data needed.

Retrieving excess data and retrieving missing data.

REST APIs always return a complete dataset. For example, from an object.personIn a REST API, you will get the person's name, date of birth, address, and phone number. You will get all this data even if you only need the phone number.

Similarly, if you want to know a person's phone number and their most recent purchase, you'll need multiple REST API requests. URL/personIt will return the phone number and URL./purchaseThis will return your purchase history.

Social media developers have had to write a lot of code just to handle API requests, which impacts performance and user experience.

GraphQL emerged as a query-based solution. Queries can return accurate data in just one API request and response exchange.

The main difference between GraphQL and REST

REST API is an architectural concept for application communication. GraphQL, on the other hand, is a specification, an API query language, and a toolkit. GraphQL operates on a single endpoint using HTTP.

Furthermore, the REST development process focuses more on creating new APIs, while GraphQL focuses on API performance and flexibility.

Next, we will point out some more differences.

Client-side request

Here are the things a REST request uses to function:

- HTTP verbs define actions.
- A URL specifies the resource to which the action will be performed according to the HTTP verb.
- Parameters and values for parsing, if you want to create or modify an object in an existing server-side resource.

For example, you useGETTo obtain read-only data from a resource,POSTto add a new resource item, orPUTto update a resource.

Conversely, these are the things that GraphQL requests use:

- Query to retrieve read-only data
- Transform to modify data
- Sign up to receive event-based or stream-based data updates.

Data format This describes how you want the server to return data, including objects and fields that match the server-side schema. You can also import new data. Internally, GraphQL sends every client request as an HTTP request. POST.

Data returned to the client

In the REST architecture, data is returned to the client from the server in a “resource-wide” structure specified by the server. The following examples illustrate data return in REST and GraphQL.

Examples of data returned in REST

In REST, GET /posts It returns the following:

```
[
  {
    "id": 1,
    "title": "First Post",
    "content": "This is the content of the first post."
  },
  {
    "id": 2,
    "title": "Second Post",
    "content": "This is the content of the second post."
  },
  {
    "id": 3,
    "title": "Third Post",
    "content": "This is the content of the third post."
  }
]
```

Examples of data returned in GraphQL

When you use GraphQL, only the data specified in the client-provided structure is returned.

GET /graphql?query{post(id: 1) {id title content}} cReturn only the first post:

```
{
```

```
"data": {
  "posts": [
    {
      "id": "1",
      "title": "First Post",
      "content": "This is the content of the first post."
    },
  ]
}
```

Server-side diagram

GraphQL uses a server-side schema to define data and data services, which is different from the REST API.

The schema was written in the GraphQL schema definition language and includes the following details:

- The types and fields of objects that belong to each object.
- The server-side resolver function defines an operation for each field.

The diagram clearly defines type to describe all the data available on the system and how clients can access or modify that data.

Meanwhile, REST APIs don't require a server-side schema. However, you can choose to define a schema to design APIs, document, and develop clients efficiently.

Versioning

As APIs evolve, their data structures and manipulations may change. For clients who are unaware of these changes, these changes could break the client's system or cause unknown errors.

REST APIs often include URL versioning to address this issue, such as `https://example.com/api/v1/person/12341`. However, versioning is not mandatory and can cause errors.

GraphQL requires backward API compatibility. Therefore, deleted fields return an error message, as do fields with `tags.deprecated` (No longer recommended for use) returns a warning.

Error handling

GraphQL is a strongly typed API architecture, meaning it requires detailed descriptions of data, data structures, and data operations within the schema. Due to this level of detail in the schema, the system can automatically identify request errors and provide helpful error messages.

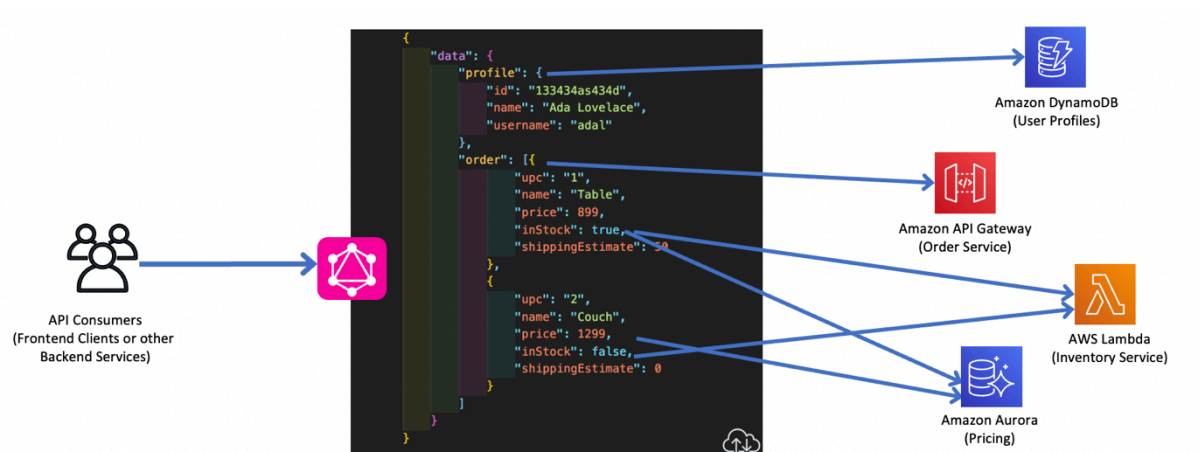
REST APIs are weakly styled, and you have to build error handling procedures into the surrounding code. For example, if a request...PUTWhen parsing a numerical value as text instead of an integer, the system will not automatically detect an error.

Comparing when to use GraphQL or REST.

You can use GraphQL and REST APIs interchangeably. However, in some cases, one API will be more suitable than the other.

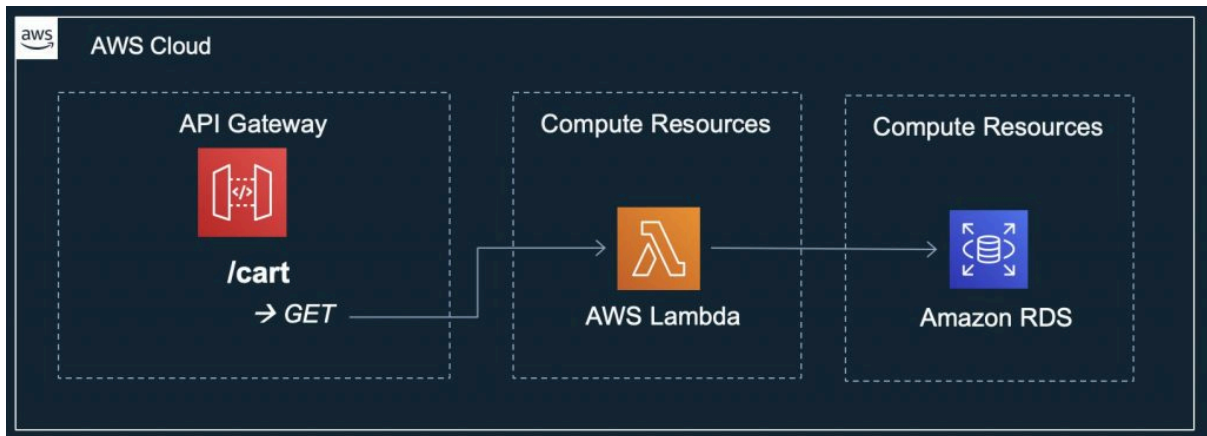
For example, GraphQL might be a better choice if you have the following considerations:

- You have limited bandwidth and you want to minimize the number of requests and responses.
- You have multiple data sources and you want to combine them at a single endpoint.
- You have client requests that differ significantly, and you want very different responses.



Meanwhile, REST might be a better choice if you have the following considerations:

- You have smaller applications with less complex data.
- You have data and operations that all clients use in the same way.
- You have no requirements for complex data queries.



How to use both GraphQL and REST on the same API

It is possible to upgrade a RESTful API to a GraphQL API without completely rewriting it.

Here's an outline of the process:

1. Understand the data model of a RESTful API. To do this, examine the data structure within each URL resource.
2. Write a GraphQL schema from the data model.
3. Identify which operations the client performs on the data and add those operations to the schema.
4. Implement a resolver function in the server-side code for each field in the schema.
5. Create a GraphQL server with resolvers and schemas.

The client can then communicate with your API using GraphQL or REST.